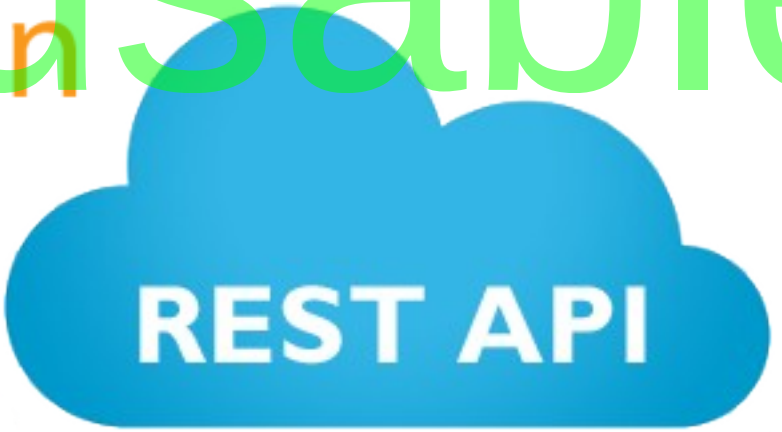




non diffusable





Définitions

- Une API (Application Programming Interface) est un ensemble de services fournis par une application en vue de répondre à des sollicitations externes.
- Une API REST (Representational State Transfer) est une API stateless (sans état). Cela veut dire que chaque appel à l'API doit contenir toutes les informations nécessaires à sa résolution. Il n'y a pas de contexte serveur pris en compte.
- On accède à une API REST via le protocole HTTP. Les différentes entités du modèle sont accédées selon l'URL utilisée alors que les actions CRUD à exécuter sont déterminées par les méthodes HTTP.

non diffusable



Méthodes d'accès

- Vous avez déjà côtoyé, en développant des sites web, les méthodes HTTP **GET** (celle utilisée pour ramener une page dans le navigateur) et **POST** (celle utilisée pour soumettre un formulaire au serveur)
- Les différentes méthodes HTTP utilisées selon le CRUD sont les suivantes :

non diffusable

Méthode CRUD	Methode HTTP	Exemple d'URL
Create	POST	/api/users/create
Read	GET	/api/users/readAll pour renvoyer toutes les données /api/users/{idUser} pour renvoyer un seul enregistrement.
Update	PUT	/api/users/update/
Delete	DELETE	/api/users/delete/{idUser}



Gestion de séries : API

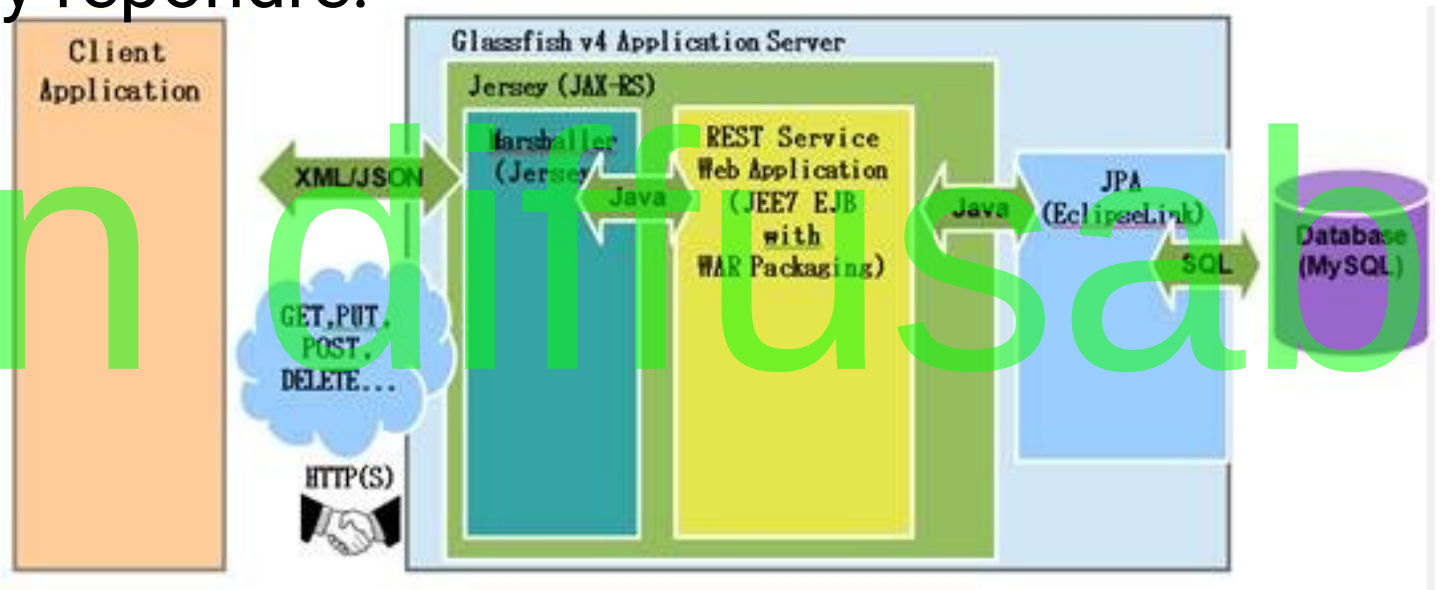
- Nous allons modifier le projet de gestion de séries afin qu'en plus d'être capable d'afficher les informations via des pages jsp, il puisse aussi fournir une API.
- Pour gérer l'API REST, nous allons utiliser le framework « Jersey » et, entre autre, son module permettant de gérer le format json.
- Grâce à Maven, ajoutez les dépendances suivantes à votre projet (en version 3.0.1) :
  - jersey-container-servlet-core ;
  - jersey-hk2 ;
  - jersey-media-json-jackson ;
  - jersey-media-json-binding.

non diffusable



Gestion de séries : API

- Une API « à la mode » Jersey consiste en un unique servlet chargé de router les requêtes HTTP sur des « services » chargés d'y répondre.



non diffusable

- Ce servlet étant fourni par « Jersey », nous allons juste le déclarer et le paramétrer dans le fichier « web.xml »

Gestion de séries : API

- Voilà ce qu'il faut rajouter dans le web.xml :

```
<servlet>
    <servlet-name>RestServer-serlvet</servlet-name>
    <servlet-class>org.glassfish.jersey.servlet.ServletContainer</servlet-class>
    <init-param>
        <param-name>jersey.config.server.provider.packages</param-name>
        <param-value>fr.pau.univ.series.services</para
    </init-param>
    <load-on-startup>1</load-on-startup>
</servlet>
<servlet-mapping>
    <servlet-name>RestServer-serlvet</servlet-name>
    <url-pattern>/api/*</url-pattern>
</servlet-mapping>
```

Même nom de servlet.

Classpath du servlet jersey.

Ces deux lignes disent à Jersey dans quel package aller chercher les services

Url mappée vers les services.

non diffusable



Rest API : les services

- Un service est défini comme une méthode dans une classe sans héritage particulier.

- Ce qui va faire que jersey va considérer les méthodes d'une classe comme des services de l'API, ce sont :

  - Le package dans laquelle est contenu la classe (celui spécifié dans le fichier web.xml grâce au paramètre **`jersey.config.server.provider.packages`**)

  - Des annotations spécifiques à jersey.

non diffusable



Rest API : les services

- La première annotation se situe au niveau de la définition de la classe :

```
@Path("/serie")
public class SeriesService {
```

Ce path sera à ajouter à l'url pour accéder aux services fournis par la classe. (« **.../serie** »)

non diffusable

- Les suivantes, au niveau de chacune des méthodes publiques destinées à renvoyer des informations aux clients de l'API :

```
@GET
@Path("/all")
@Produces(MediaType.APPLICATION_JSON)
public Response getSeries() {
```

Type de requête

Path additionnel du service (« **.../serie/all** »)

Type de données renvoyées



Rest API : les services

- Ainsi, la méthode **`getSeries`** définie ci dessous ...

```
@Path("/serie")
public class SeriesService {

    @GET
    @Path("/all")
    @Produces(MediaType.APPLICATION_JSON)
    public Response getSeries() {
```

non diffusable

- ... sera accédée par une requête de type GET ...
- ... à l'URL **`host:port/GestionSeries/api/serie/all`**

« **display-name** » dans web.xml

« **url-pattern** » dans web.xml

Annotation « **@Path** » de la classe

Annotation « **@Path** » de la méthode



Rest API : Tester l'API

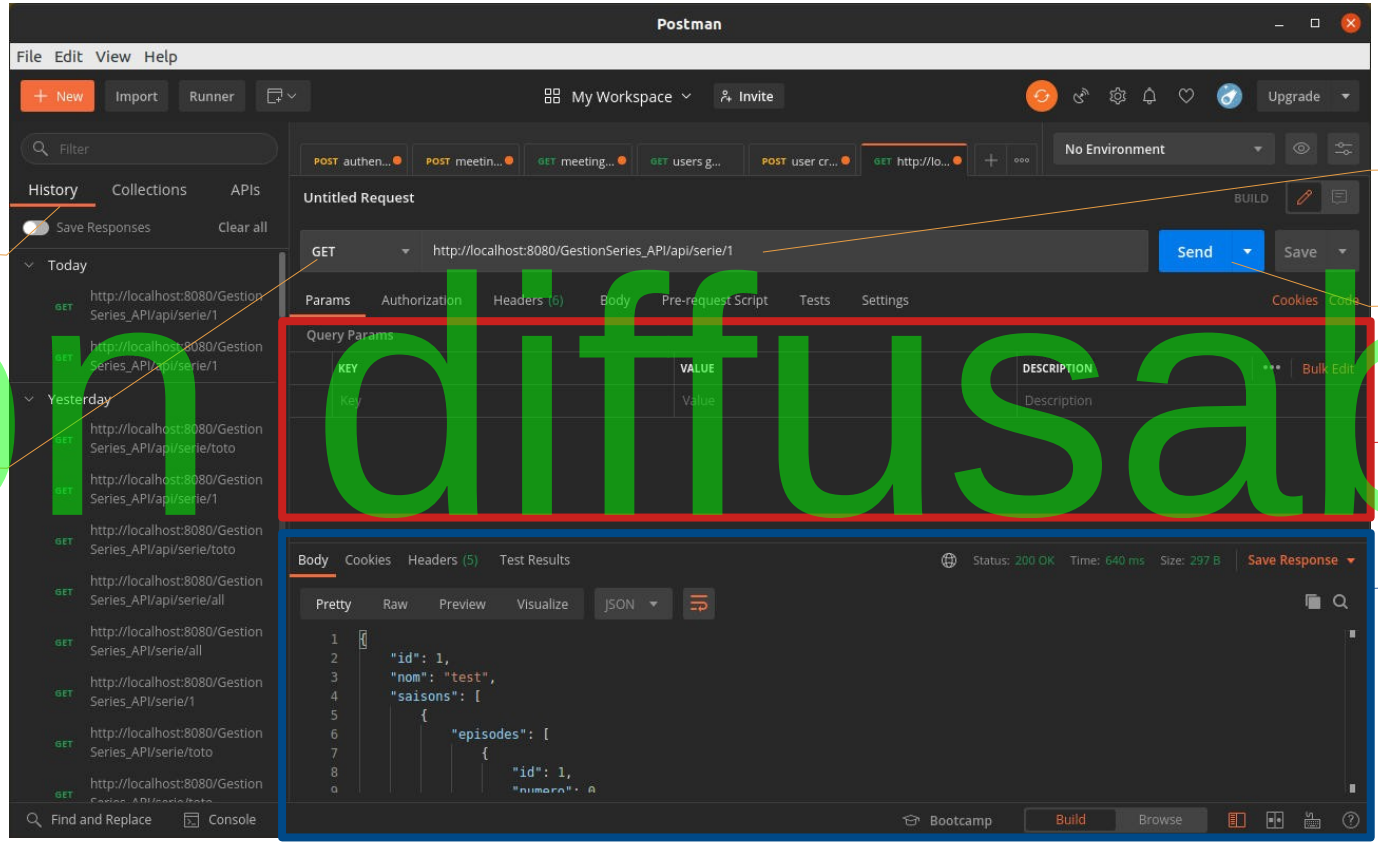
- Pour pouvoir tester nos API, un navigateur internet ne suffit pas. En effet, mis à part le « **`GET`** », les autres types de requêtes ne sont pas facile à tester.
- Il existe des outils spécialement prévus pour ça. Les plus connus sont *CURL* et *Postman*. C'est ce deuxième que je vous propose d'utiliser.
- Postman peut être téléchargé à l'adresse suivante : « **`https://www.postman.com/downloads/`** ».
- Sélectionnez la version correspondant à vôtre OS, téléchargez-la et installez-la.
- Il vous faudra ensuite vous inscrire pour pouvoir utiliser l'outil.

non diffusable



Rest API : Tester l'API

- L'interface utilisateur de Postman est la suivante :



Historique des requêtes testées et gestion de « collections » de requêtes

Choix du type de requête à tester

Url de la requête à tester.

Bouton « Send » pour lancer le test

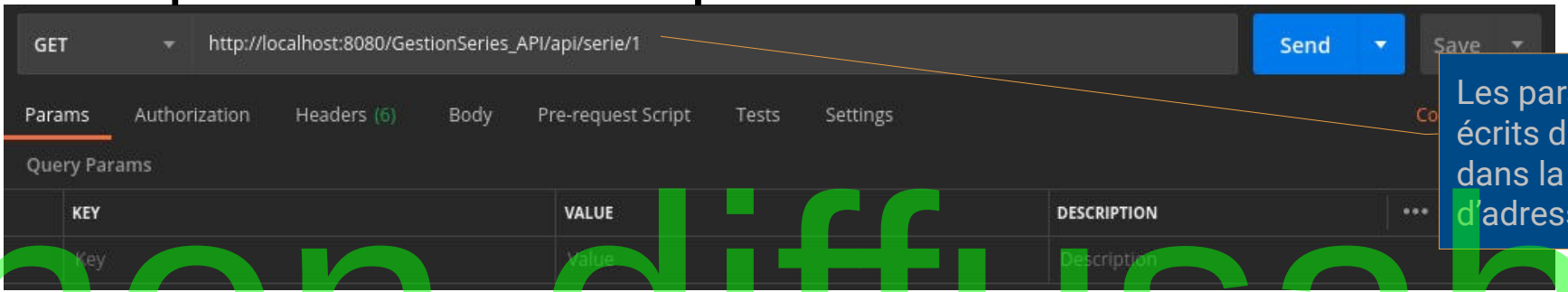
Bloc des paramètres

Bloc d'affichage du résultat.

non diffusable

Rest API : Tester l'API

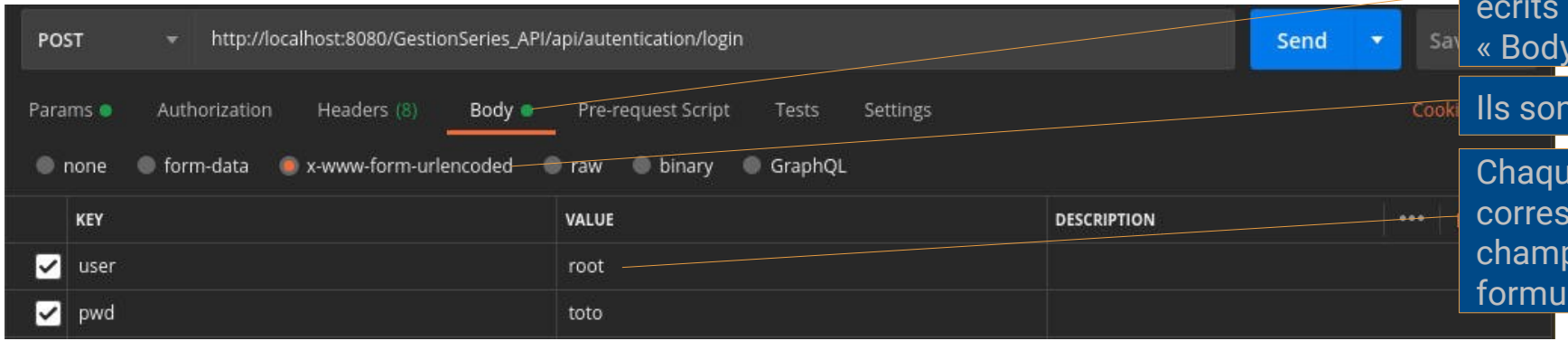
- Passer les paramètres à notre requête variera selon le type de requête. Pour les requêtes **GET** et **DELETE** :



Les paramètres sont écrits directement dans la barre d'adresse….

non diffusable

- Les autres types de requêtes recevront leurs paramètres d'un formulaire encodé:



Les paramètres sont écrits dans l'onglet « Body »

Ils sont encodés.

Chaque ligne correspond à un champ dans un formulaire

Rest API : passage de paramètres

- Si la requête est de type **GET** ou **DELETE**, les paramètres peuvent être passés directement dans l'URL :

```
@GET
@Path("/{id}")
@Produces(MediaType.APPLICATION_JSON)
public Response getSerieById(@PathParam("id") final int idSerie) {
```

Même nom

non diffusable

- Cette méthode répondra à une requête **GET** de type : **host:port/GestionSeries/api/serie/42**
- Mais : **host:port/GestionSeries/api/serie/toto** ne fonctionnera pas parce que le paramètre attendu est de type entier.
- On peut chainer les paramètres par une annotation du type : **@Path("/{name}/{type}")**



Rest API : passage de paramètres

- Pour les autres types de requêtes, on utilise un formulaire dont le type va être spécifié par une annotation :

```
@POST
@Consumes("application/x-www-form-urlencoded")
@Path("/add")
@Produces(MediaType.APPLICATION_JSON)
public Response addSerie(final MultivaluedMap<String, String> formParams) {
```

L'annotation **@Consumes** précise le type de paramètre en entrée du service.

non diffusable

- Les paramètres sont reçus dans la méthode Java sous la forme d'une **MultivaluedMap** dans laquelle :
  - La clé est le nom du paramètre
  - La valeur est un tableau des différentes valeurs qu'a le paramètre portant ce nom (HTTP autorisant plusieurs champs avec le même nom)



Rest API : passage de paramètres

- On utilise donc les paramètres reçus de cette façon :

```java
@POST
@Path("/login")
@Consumes("application/x-www-form-urlencoded")
@Produces("application/json")
public Response autenticateUser(final MultivaluedMap<String, String> formParams) {
    if (formParams.get("user") == null || formParams.get("pwd") == null) {
        return Response.status(Response.Status.FORBIDDEN).build();
    }
    final String user = formParams.get("user").get(0);
    final String pwd = formParams.get("pwd").get(0);
    if (user == null || user.isBlank() || pwd == null || pwd.isBlank()) {
        return Response.status(Response.Status.FORBIDDEN).build();
    }
    try { //...
```

On teste d'abord la **présence** des différents paramètres attendus.

On peut aussi utiliser **`formParams.getFirst("user")`**

Puis, on teste leur contenu.

non diffusable



Rest API : les services

- Maintenant que nous savons quelles signatures vont avoir nos méthodes de service, il faut les remplir.
- Les plus simples sont les méthodes **GET** sans paramètres. Globalement, on laisse Jersey transformer nos objets en JSON :

non diffusable

```
@GET
@Path("/all")
@Produces(MediaType.APPLICATION_JSON)
public Response getSeries() {
    try {
        final List<Serie> ret = DaoFactory.getInstance().getSerieDao().readAllSeries();
        final GenericEntity<List<Serie>> json = new GenericEntity<>(ret) {};
        return Response.ok().entity(json).build();
    } catch (final DaoException e) {
        return Response.status(Response.Status.INTERNAL_SERVER_ERROR)
            .entity(e.getMessage()).build();
    }
}
```

On encapsule notre retour dans une **GenericEntity** typée avec le retour désiré.
Notez le « **{}** » en fin de ligne : c'est une création de classe anonyme.

Statut **OK** si tout s 'est bien passé

Statut **INTERNAL_SERVER_ERROR** en cas d'exception.

Les statuts HTTP

- Pour rappel, les statuts HTTP sont des entiers à 3 chiffres dont la centaine donne la famille. En JEE, ils sont encapsulés dans l'enum **`Response.Status`**. Résumé des plus courants:
  - **HTTP 2xx : Success**. On utilise couramment
    - le 200 (**`OK`**) pour la majorité des requêtes ;
    - le 201 (**`CREATED`**) permettra de spécifier au client de l'API qu'une création s'est bien déroulée
  - **HTTP 4xx : Error**. Une API utilisera majoritairement :
    - Le 400 (**`BAD_REQUEST`**) si les paramètres ne sont pas OK.
    - Le 403 (**`FORBIDDEN`**) si l'utilisateur n'a pas le droit d'accès à la méthode demandée;
    - le 404 (**`NOT_FOUND`**) si rien n'a été trouvé ;

non diffusable



Les statuts HTTP (suite)

- Les derniers status sont ceux concernant les erreurs :
  - **HTTP 5xx : Serveur error**. Erreur en provenance du serveur.
    - On utilisera l'erreur 500 (**`INTERNAL_SERVER_ERROR`**) à chaque fois qu'une exception sera levée dans le code Java de nos services.
    - Les autres valeurs sont des status retournés directement par Tomcat en cas de problème.

non diffusable

- Pour plus de détails sur les statuts HTTP :
  `http://www.w3.org/Protocols/rfc2616/rfc2616-sec10.html`



Rest API : les services

- Pour les méthodes type **`getXXXbyId`**, il faut spécifier si on a trouvé le *XXX* demandé :

```
@GET
@Path("/{id}")
@Produces(MediaType.APPLICATION_JSON)
public Response getSerieById(@PathParam("id") final int idSerie) {
    Serie s = null;
    try {
        s = DaoFactory.getInstance().getSerieDao().readSerie(idSerie);
        if (s == null) {
            return Response.status(Response.Status.NOT_FOUND)
                    .entity("Serie not found").build();
        } else {
            final GenericEntity<Serie> json = new GenericEntity<>(s) {};
            return Response.ok().entity(json).build();
        }
    } catch (final DaoException e) {
        return Response.status(Response.Status.INTERNAL_SERVER_ERROR)
                .entity(e.getMessage()).build();
    }
}
```

Statut **NOT_FOUND** si aucune série n'a été trouvée avec l'Id fourni.

Statut **OK** si tout s 'est bien passé

Statut **INTERNAL_SERVER_ERROR** en cas d'exception avec le message d'erreur en contenu.

non diffusable

Rest API : les services

- Pour les autres méthodes, on vérifie les paramètres :

```
public Response addEpisode(final MultivaluedMap<String, String> formParams) {
    if (formParams.get("titre") == null || formParams.get("numero") == null ||
            formParams.get("saisonId") == null) {
        return Response.status(Response.Status.BAD_REQUEST).entity("Invalid parameters").build();
    } else {
        final Saison s = null;
        try {
            s = DaoFactory.getInstance().getSaisonDao()
                .readSaison(Integer.parseInt(formParams.getFirst("saisonId")));
        } catch (final DaoException e) {
            return Response.status(Response.Status.INTERNAL_SERVER_ERROR)
                .entity(e.getMessage()).build();
        } catch (final NumberFormatException e) {
            return Response.status(Response.Status.BAD_REQUEST)
                .entity("saisonId must be an intger").build();
        }
        if (s == null) {
            return Response.status(Response.Status.NOT_FOUND)
                .entity("Parent season not found").build();
        }
        // Suite de la méthode avec statut CREATED si OK, INTERNAL_SERVER_ERROR sinon
    }
}
```

Statut **BAD_REQUEST** si un parametre obligatoire n'a pas été fourni.

Statut **INTERNAL_SERVER_ERROR** en cas d'exception avec le message d'erreur en contenu.

Statut **BAD_REQUEST** si un parametre obligatoire n'est pas du bon type.

Statut **NOT_FOUND** si l'Id fourni ne correspnd à aucune saison.

non diffusable

Rest API : la sécurité

- Dans de nombreux cas, une API ne peux rester « ouverte aux 4 vents ».
- Elle va comporter, en général, certains services « tout public » et d'autres pour lesquels l'accès sera filtré selon le profil d'un utilisateur authentifié.
- Une bonne façon de sécuriser une API est d'utiliser un « jeton » (token).
- Celui-ci est renvoyé par le service de login et peut être comparé à la session dans un site web : il est comporte des informations sur le profil et peut être limité dans le temps.

non diffusable



Rest API : Utilisation de JWT

- Il existe un standard très utilisé pour ce type d'usage appelé JWT (pour Json Web Token).
- Ce standard a été implémenté en Java et nous allons nous en servir plutôt pour sécuriser notre API.
- Allez sur Maven Repository et récupérez la dépendance « jjwt » (Java JWT) pour l'ajouter aux dépendances de votre projet.

non diffusable

```
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt</artifactId>
    <version>0.9.1</version>
</dependency>
```



Rest API : Fonctionnement d'un token

- Le principe de fonctionnement d'une sécurité à base de token est le suivant :
  1) Le **service d'authentification** (qui est « tout public ») reçoit le « login/mdp » de l'utilisateur et vérifie la présence de l'utilisateur en base. Il récupère éventuellement son profil. Il faudra donc rajouter un objet User ainsi que le DAO associé.
  2) Si l'utilisateur est autorisé, un token est généré qui contient des informations sur le profil éventuel et une date/time de péremption. Ce token est renvoyé par le service.
  3) Le token est passé (en plus des paramètres fonctionnels) aux services à sécuriser afin d'autoriser l'accès.

non diffusable



Rest API : Fonctionnement d'un token

- Réception du « login/mdp » de l'utilisateur et lecture BDD.

```
public Response autenticateUser(final MultivaluedMap<String, String> formParams) {
    if (formParams.get("user") == null || formParams.get("pwd") == null) {
        return Response.status(Response.Status.BAD_REQUEST)
                .entity("Invalid parameters").build();
    }
    final String login = formParams.get("user").get(0);
    final String pwd = formParams.get("pwd").get(0);
    if (login == null || login.isBlank() || pwd == null || pwd.isBlank()) {
        return Response.status(Response.Status.BAD_REQUEST)
                .entity("Invalid parameters").build();
    }
    try {
        final User usr = DaoFactory.getInstance().getUserDao() .readUserByLogin(login);
        if (usr == null || !PasswordHelper.verifyUserPassword(pwd, usr.getPassword())) {
            return Response.status(Response.Status.BAD_REQUEST)
            .entity("Unknown user or bad password").build();
        }
        final String token = createJWT(usr, TOKEN_LIFETIME_MS);
        return Response.ok(token).build();
    } catch (final DaoException e) {
        return Response.status(Response.Status.INTERNAL_SERVER_ERROR).build();
    }
}
```

Le mot de passe étant crypté en base, on a codé une classe utilitaire permettant de gérer son cryptage / décryptage.

Statut **BAD_REQUEST** si l'utilisateur n'a pas été trouvé ou que le mdp n'est pas bon.

Durée de vie du token en ms (constante)

non diffusable

Rest API : Fonctionnement d'un token

- Génération du Token.

```
private String createJWT(final User user, final long ttlMillis) {

    final byte[] apiKeySecretBytes = DatatypeConverter.parseBase64Binary(SECRET_API_KEY);
    final SignatureAlgorithm signatureAlgorithm = SignatureAlgorithm.HS256;
    final Key signingKey = new SecretKeySpec(apiKeySecretBytes,
                signatureAlgorithm.getJcaName());

    final long nowMillis = System.currentTimeMillis();
    final Date now = new Date(nowMillis);

    final String id = UUID.randomUUID().toString();
    final JwtBuilder builder = Jwts.builder().setId(id)
                .setIssuedAt(now)
                .setSubject(user.getRole().name())
                .setIssuer(user.getLogin()).signWith(signatureAlgorithm, signingKey);

    final long expMillis = nowMillis + ttlMillis;
    final Date exp = new Date(expMillis);
    builder.setExpiration(exp);
    return builder.compact();
}
```

La clé secrète de notre API esr stockée dans une constante de type String/

Le token est protégé par une signature cryptée.

Récupération de la date système du serveur.

Le token est horodaté

Le profil de l'utilisateur est stocké dans le sujet du token

Le login de l'utilisateur est stocké dans l'émetteur du token

On rajoute une expiration au token avant de le renvoyer.

non diffusable

Rest API : Vérification du token

- Dans les services à sécuriser, il suffira ensuite, avant toute autre action, d'appeler la méthode **checkToken** pour récupérer le login et le profil de l'utilisateur.

```
try {
    User usr = AuthenticationService.checkToken(formParams);
    if (!usr.getRole().equals(UserRole.ROLE_ADMIN)) {
        return Response.status(Response.Status.FORBIDDEN)
                .entity("Only admin can do this").build();
    }
} catch (final ServiceException e) {
    return Response.status(Response.Status.FORBIDDEN).entity(e.getMessage()).build();
}
```

Exemple de vérification de profil

Cette exception est levée si le token est absent ou invalide.

non diffusable

- Ne pas oublier, lors de l'appel à ces services de rajouter le paramètre « token » au formulaire avec, comme valeur, le token (chaîne de caractères) reçu lors de l'authentification.



Rest API

- Dans **AuthenticationService**

```java
public static User checkToken(final MultivaluedMap<String, String> formParams) throws ServiceException {
    if (formParams.get("token") == null) {
        throw new ServiceException("Vous devez fournir un token d'identification valide");
    }
    final String token = formParams.getFirst("token");
    if (token == null || token.isBlank()) {
        throw new ServiceException("Vous devez fournir un token d'identification valide");
    }
    Claims claims = null;
    try {
        claims = Jwts.parser().setSigningKey(DatatypeConverter.parseBase64Binary(SECRET_API_KEY))
                .parseClaimsJws(token).getBody();
    } catch (final Exception e) {
        throw new ServiceException("Token invalide");
    }
    final Date now = new Date(System.currentTimeMillis());
    if (now.after(claims.getExpiration())) {
        throw new ServiceException("La durée de validité du Token est expirée.");
    }
    final User ret = new User();
    ret.setLogin(claims.getIssuer());
    ret.setRole(UserRole.valueOf(claims.getSubject()));
    return ret;
}
```

Décryptage du token et extraction de son contenu.

Vérification de l'expiration

Renvoi d'un objet User initialisé avec les données contenues dans le token.

non diffusable

Annexe : PasswordHelper

```java
public class PasswordHelper {

	private final static String	SALT		= "EqdmPh53c9x33EygXpTpcoJvc4VXLK";
	private static final int	ITERATIONS	= 10000;
	private static final int	KEY_LENGTH	= 256;

	/**
	 * Génerates an encrypted password from a clear one.
	 *
	 * @param password the readable form of the password.
	 * @return the encrypted version of given password.
	 */
	public static String generateSecurePassword(final String password) {
		String returnValue = null;
		final byte[] securePassword = hash(password.toCharArray(), SALT.getBytes());
		returnValue = Base64.getEncoder().encodeToString(securePassword);
		return returnValue;
	}

	/**
	 * Checks if given clear and encrypted password match.
	 *
	 * @param providedPassword unencrypted version of password
	 * @param securedPassword encrypted version of password
	 * @return <code>true</code> if given clear and encrypted password match, <code>false</code> otherwise.
	 */
	public static boolean verifyUserPassword(final String providedPassword, final String securedPassword) {
		final String newSecurePassword = generateSecurePassword(providedPassword);
		return newSecurePassword.equals(securedPassword);
	}

	private static byte[] hash(final char[] password, final byte[] salt) {
		final PBEKeySpec spec = new PBEKeySpec(password, salt, ITERATIONS, KEY_LENGTH);
		Arrays.fill(password, Character.MIN_VALUE);
		try {
			final SecretKeyFactory skf = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA1");
			return skf.generateSecret(spec).getEncoded();
		} catch (NoSuchAlgorithmException | InvalidKeySpecException e) {
			throw new AssertionError("Error while hashing a password: " + e.getMessage(), e);
		} finally {
			spec.clearPassword();
		}
	}
```

non diffusable